Document Summary

This note provides an introduction to Reinforcement Learning (RL), covering its fundamental concepts, information representation, model selection, probabilistic learning, and key algorithms. It also discusses neural networks, activation functions, loss functions, optimization algorithms, and regularization techniques, along with deep learning models and attention mechanisms.

Intro to Reinforcement Learning

Overview of Machine Learning (ML)

Machine Learning (ML): A field of artificial intelligence dealing with models and methods that allow computers to learn from data.

ML Tasks & Data:

- **Supervised Learning:** Learn an unknown function predicting an output in response to an input.
  - Example: Predicting credit risk given customer profile.
- **Unsupervised Learning:** Identification of structures, regularities, associations, and anomalies in the data.
  - Example: Signaling anomalous transactions.
- **Reinforcement Learning:** Learning of a policy or complex behaviour while being allowed to observe only partial responses from the interaction with the environment or the user.
  - Example: Autonomous agents (s, a, r).

Information Representation in ML

Input Sample: The i-th input sample x_i is a D-dimensional numerical vector.

- Continuous, categorical, or mixed values.

- Describes an individual of our world of interest, e.g., patients in a biomedical application.

Output Sample: y_i is a D'-dimensional numerical vector.

Data Types:

- **Images:** Matrices of pixel intensity.
- **Structured Data:** Relational information comprising atomic elements that needs to be interpreted in the context of the surrounding elements.
- **Sequential Data:** Variable size data characterized by sequentially dependent information.
  - Examples: Financial time series, sequences of operations, natural language sentences.

Fundamental Concepts in ML

ML Model: Computational model $M_\alpha(D, \theta)$ that can be applied to data D and whose behavior is regulated by adaptive parameters θ and by hyperparameters α (externally set).

Training: Process through which model M parameters θ are modified to adapt to training data D_{Tr} by optimizing a cost function $E(\theta, D_{Tr})$.

Generalization: Sought property of a model M that, trained on D_{Tr}, generalizes well its output on new/fresh data D_{Test}.

Overfitting: Problem inducing poor generalization in a trained model, which behaves excellently on training data while being very poor on test.

Model Selection

Set of techniques from robust statistics to measure generalization, avoid overfitting, and reduce the effect of model bias.

1. **Separate training phase, from the choice of model configuration (including hyperparameters), from model generalization assessment.**
   - Training, Validation, Testing Data
2. **Iterate the process changing data to obtain robust performance estimates.**
   - k-fold validation

Probabilistic Learning

A general probabilistic learning model comprises:

- Observable random variables X (data)
- Hidden random variables Z (latent)
- Model parameters θ

Likelihood: $P(X|Z,\theta)$

Prior: $P(Z|\theta)$

Posterior:

$$P(Z|X,\theta) = \frac{P(X|Z,\theta)P(Z|\theta)}{P(X|\theta)}$$

Marginal:

$$P(X|\theta) = \int P(X|Z,\theta)P(Z|\theta)dz$$

Maximum Likelihood Learning

Find model parameters by maximizing model likelihood.

Expectation-Maximization (EM) Algorithm:

- (E) Given the current model parameters θ^k, compute the posterior expectation.
- (M) Given the current posterior expectation, update parameters $\theta^k = \arg\max Q(\theta|\theta^k)$

$$\theta^* = \arg\max \log P(X|\theta) = \arg\max \log \int P(X|Z,\theta)P(Z|\theta)dz$$

$$Q(\theta|\theta^k) = \mathbb{E}_{Z|X,\theta^k} \log P(X,Z|\theta)$$

Evidence Lower Bound (ELBO)

Posterior is not always easily computable or available in closed-form, so we minimize a lower bound with respect to a variational distribution $Q(Z|\lambda)$ with parameters λ.

$$\log P(X|\theta) \geq \mathbb{E}_{Z|\lambda} \log P(X,Z|\theta) - \mathbb{E}_{Z|\lambda} \log Q(Z|\lambda) = \mathcal{L}(X,\theta,\lambda)$$

Sampling Approximations

- Ancestral sampling
- Gibbs sampling
- Markov Chain Monte Carlo Methods
- Importance sampling (particle filtering)

Fundamentals of Neural Networks

Neural Networks and Inductive Bias:

- Architectural design influences deeply the type of tasks it can solve, the type of data it can handle, and the quality of generalization of its results.
- Architectural choices: Topology and weight sharing, activation functions, regularization strategies, loss functions.

Logistic Neuron:

$$\theta_1, \theta_2, \theta_3$$

Multilayer Perceptron (Single Output):

- Input
- Hidden Layer
- Output

Multilayer Perceptron (Multi-class output):

- Input
- Hidden Layer
- Output

Activation Functions

- $f(x) = x$
- $f(x) = \frac{1}{1+e^{-x}}$ (Sigmoid)
- $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ (Hyperbolic Tangent)
- $f(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$ (Step Function)
- $f(x) = \begin{cases} 0 \text{ or } \epsilon, & x < 0 \\ x, & x \geq 0 \end{cases}$ (Approximate Step Function)

Training Neural Networks

Gradient Descent:

- Weights are updated in the opposite direction of the gradient of the loss function.
- Gradient can be backpropagated by the chain rule.

Loss Functions for Neural Networks

Regression:

- Output Layer: One node with a linear activation unit.
- Loss Function: Quadratic Loss (Mean Squared Error (MSE))

$$J = \frac{1}{2}(y - y^*)^2$$

$$\frac{dJ}{dy} = y - y^*$$

Classification:

- Output Layer: One node with a sigmoid activation unit ($K = 2$) or K output nodes in a softmax layer ($K > 2$).
- Loss Function: Cross-entropy (i.e., negative log likelihood)

$$J = y^* \log y + (1 - y^*) \log(1 - y)$$

Optimization Algorithms

- **Standard Stochastic Gradient Descent (SGD):** Easy and efficient but difficult to pick up the best learning rate.
- **RMSprop:** Adaptive learning rate method (reduces it using a moving average of the squared gradient).
- **Adagrad:** Like RMSprop with element-wise scaling of the gradient.
- **ADAM:** Like Adagrad but adds an exponentially decaying average of past gradients like momentum.

Learning Fashions

- **Sequential mode (on-line, stochastic, or per-pattern):** Weights updated after each pattern is presented.

- **Batch mode (off-line or per-epoch):** Weights updated after all patterns are presented.
- **Minibatch mode (a blend of the two above):** Weights updated after a few patterns.

Convergence Criteria

- Euclidean norm of the gradient vector reaches a sufficiently small value.
- Absolute rate of change in the average squared error per epoch is sufficiently small.
- Validation for generalization performance: stop when generalization performance reaches a peak.

Regularization

Constrain the learning model to avoid overfitting and help improving generalization.

$$J' = J(y, y^*) + \lambda R(\cdot)$$

Common penalty terms (norms):

- 1-norm: $||A||_1 = \sum_{ij} |a_{ij}|$
- 2-norm: $||A||_2 = \sqrt{\sum_{ij} a_{ij}^2}$

Dropout Regularization

- Regulated by unit dropping hyperparameter.
- Prevents unit coadaptation.
- Committee machine effect.
- Used at prediction time gives predictions with confidence intervals.
- Dropconnect: drops single connections.

Deep Learning Models

Deep Neural Networks:

- Input
- Hidden Layer 1
- Hidden Layer 2
- Output
- Hidden Layer 3

Representation Learning:

- Input
- Hidden Layer 1
- Hidden Layer 2
- Output
- Hidden Layer 3

Autoencoders

Basic Autoencoder (AE):

- Input
- Hidden
- Latent space
- Output

Deep Autoencoder:

- Input
- Hidden Layer 1
- Hidden Layer 2
- Hidden Layer 3
- Output

Variational Autoencoder (VAE):

- $P(z|x)$
- $P(x|z)$
- Reparameterization trick

Convolutional Neural Networks

Adaptive Convolution Operator:

- Convolutional filter (kernel) with (adaptive) weights w_i
- Feature map transformation

Pooling:

- Max pooling
- 2x2 filters, stride = 2

Recurrent Neural Networks (RNNs)

Unfolding RNN (Forward Pass):

- Input sequence
- Hidden state
- Output

Recurrent Neural Network:

- σ
- x_t
- h_t
- h_{t-1}
- $W_i x_t$
- $W_h h_{t-1}$
- g_i
- $h_t = \tanh(g_t)$
- $y_t = f(W_{out} h_t + b_{out})$

Learning to Encode Input History:

- Hidden state h_t summarizes information on the history of the input signal up to time t.

Learning Long-Term Dependencies is Difficult:

- Exploding/vanishing gradient

Gated Recurrent Networks:

- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)

Encoder-Decoder Architectures

Encoder RNN:

- Encodes input sequence into a fixed size vector and then is passed to decoder RNN.

Attention

- $h_1, h_2, h_3, \ldots, h_n$
- Context s
- $e_1, e_2, e_3, \ldots, e_n$
- $\alpha_1, \alpha_2, \alpha_3, \ldots, \alpha_n$
- Softmax

ML, DL and RL Frameworks

References:

- Ian Goodfellow and Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press
- David Barber, Bayesian Reasoning and Machine Learning, Cambridge University Press